

Lab Assignment-16.1

Name: CH VISHWANATH RAO

Hallticket-2303A51095

Batch-02

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
 - o Insert a new student record.
 - o Fetch all courses enrolled by a student.
 - o Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student–course relationships.

Screenshots:

```
ASSG-16-1.sql
1  --create tables named as students and courses and enrollments and define primary keys foreign keys and relationships
2  CREATE TABLE IF NOT EXISTS courses (
3      course_id INT PRIMARY KEY,
4      course_name VARCHAR(50) NOT NULL
5  );
6  CREATE TABLE IF NOT EXISTS enrollments (
7      enrollment_id INT PRIMARY KEY,
8      student_id INT,
9      course_id INT,
10     FOREIGN KEY (student_id) REFERENCES students(id),
11     FOREIGN KEY (course_id) REFERENCES courses(course_id)
12 );
13 CREATE TABLE IF NOT EXISTS students (
14     id INT PRIMARY KEY,
15     name VARCHAR(50) NOT NULL,
16     age INT NOT NULL,
17     grade VARCHAR(10) NOT NULL
18 );
```

```

19  --insert sample data into courses table
20  INSERT INTO courses (course_id, course_name) VALUES
21  (1, 'Mathematics'),
22  (2, 'Physics'),
23  (3, 'Chemistry');
24  --insert sample data into enrollments table
25  INSERT INTO enrollments (enrollment_id, student_id, course_id) VALUES
26  (1, 1, 1),
27  (2, 1, 2),
28  (3, 2, 1),
29  (4, 3, 3),
30  (5, 4, 2);
31  --insert sample data into students table
32  INSERT INTO students (id, name, age, grade) VALUES
33  (1, 'Alice', 20, 'A'),
34  (2, 'Bob', 22, 'B'),
35  (3, 'Charlie', 19, 'A'),
36  (4, 'David', 21, 'C'),
37  (5, 'Eve', 20, 'B');
38  --display all rows form the students table

```

Output:

SQL ▾

< 1 / 1 > 1 - 5 of 5



student_name	course_name
Alice	Mathematics
Alice	Physics
Bob	Mathematics
Charlie	Chemistry
David	Physics

SQL ▾ < 1 / 1 > 1 - 3 of 3    

course_name	student_count
Chemistry	1
Mathematics	2
Physics	2

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - o List all appointments for a specific doctor.
 - o Retrieve patient history by patient ID.
 - o Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Screenshots:

```
53 --create tables named as doctors and patients appointments and define primary keys foreign keys and realtionships contr:
54 CREATE TABLE IF NOT EXISTS doctors (
55     doctor_id INT PRIMARY KEY,
56     name VARCHAR(50) NOT NULL,
57     specialty VARCHAR(50) NOT NULL
58 );
59 CREATE TABLE IF NOT EXISTS patients (
60     patient_id INT PRIMARY KEY,
61     name VARCHAR(50) NOT NULL,
62     age INT NOT NULL
63 );
64 CREATE TABLE IF NOT EXISTS appointments (
65     appointment_id INT PRIMARY KEY,
66     doctor_id INT,
67     patient_id INT,
68     appointment_date DATE NOT NULL,
69     FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
70     FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
71 );
```

```
72 --insert sample data into doctors table
73 INSERT INTO doctors (doctor_id, name, specialty) VALUES
74 (1, 'Dr. Smith', 'Cardiology'),
75 (2, 'Dr. Johnson', 'Neurology'),
76 (3, 'Dr. Williams', 'Pediatrics'),
77 (4, 'Dr. Brown', 'Dermatology'),
78 (5, 'Dr. Davis', 'Orthopedics');
79 --insert sample data into patients table
80 INSERT INTO patients (patient_id, name, age) VALUES
81 (1, 'Alice', 30),
82 (2, 'Bob', 25),
83 (3, 'Charlie', 35),
84 (4, 'David', 28),
85 (5, 'Eve', 22);
86 --insert sample data into appointments table
87 INSERT INTO appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
88 (1, 1, 1, '2024-07-01'),
89 (2, 2, 2, '2024-07-02'),
90 (3, 3, 3, '2024-07-03'),
91 (4, 4, 4, '2024-07-04'),
92 (5, 5, 5, '2024-07-05');
```

```

94  --list all appointments for a specific doctor
95  SELECT d.name AS doctor_name, p.name AS patient_name, a.appointment_date
96  FROM appointments a
97  JOIN doctors d ON a.doctor_id = d.doctor_id
98  JOIN patients p ON a.patient_id = p.patient_id
99  WHERE d.name = 'Dr. Smith';
100  --retrive patients history by patient id
101  SELECT p.name AS patient_name, d.name AS doctor_name, a.appointment_date
102  FROM appointments a
103  JOIN doctors d ON a.doctor_id = d.doctor_id
104  JOIN patients p ON a.patient_id = p.patient_id
105  WHERE p.patient_id = 1;
106  --count total patients treated by each doctor
107  SELECT d.name AS doctor_name, COUNT(a.patient_id) AS patient_count
108  FROM doctors d
109  LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
110  GROUP BY d.name;

```

SQL ▼

< 1 / 1 > 1 - 1 of 1



doctor_name	patient_name	appointment_date
Dr. Smith	Alice	2024-07-01

SQL ▼

< 1 / 1 > 1 - 1 of 1



patient_name	doctor_name	appointment_date
Alice	Dr. Smith	2024-07-01

SQL ▼

< 1 / 1 > 1 - 5 of 5



doctor_name	patient_count
Dr. Brown	1
Dr. Davis	1
Dr. Johnson	1
Dr. Lee	1
Dr. Smith	1

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.

- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
 - o Retrieve all books currently issued.
 - o Find overdue books (loan date > 30 days).
 - o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Screenshots:

```

116  --create tables books ,members ,loans use indexxing stratagies for faster queries
117  CREATE TABLE IF NOT EXISTS books (
118      book_id INT PRIMARY KEY,
119      title VARCHAR(100) NOT NULL,
120      author VARCHAR(50) NOT NULL
121  );
122  CREATE TABLE IF NOT EXISTS members (
123      member_id INT PRIMARY KEY,
124      name VARCHAR(50) NOT NULL,
125      membership_date DATE NOT NULL
126  );
127  CREATE TABLE IF NOT EXISTS loans (
128      loan_id INT PRIMARY KEY,
129      book_id INT,
130      member_id INT,
131      loan_date DATE NOT NULL,
132      return_date DATE,
133      FOREIGN KEY (book_id) REFERENCES books(book_id),
134      FOREIGN KEY (member_id) REFERENCES members(member_id)
135  );
136  --insert sample data into books table

```

```

136  --insert sample data into all tables
137  INSERT INTO books (book_id, title, author) VALUES
138  (1, 'The Great Gatsby', 'F. Scott Fitzgerald'),
139  (2, 'To Kill a Mockingbird', 'Harper Lee'),
140  (3, '1984', 'George Orwell'),
141  (4, 'Pride and Prejudice', 'Jane Austen'),
142  (5, 'Moby Dick', 'Herman Melville');
143  INSERT INTO members (member_id, name, membership_date) VALUES
144  (1, 'Alice', '2024-01-01'),
145  (2, 'Bob', '2024-01-02'),
146  (3, 'Charlie', '2024-01-03'),
147  (4, 'David', '2024-01-04'),
148  (5, 'Eve', '2024-01-05');
149  INSERT INTO loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
150  (1, 1, 1, '2024-07-01', NULL),
151  (2, 2, 2, '2024-07-02', NULL),
152  (3, 3, 3, '2024-07-03', NULL),
153  (4, 4, 4, '2024-07-04', NULL),
154  (5, 5, 5, '2024-07-05', NULL);
155

```

```

156  --retrive all books currently issued
157  SELECT b.title, b.author, m.name AS member_name
158  FROM loans l
159  JOIN books b ON l.book_id = b.book_id
160  JOIN members m ON l.member_id = m.member_id
161  WHERE l.return_date IS NULL;
162  --find overdue books (load date>30 days )
163  SELECT b.title, b.author, m.name AS member_name
164  FROM loans l
165  JOIN books b ON l.book_id = b.book_id
166  JOIN members m ON l.member_id = m.member_id
167  WHERE l.return_date IS NULL AND DATEDIFF(CURDATE(), l.loan_date) > 30;
168  --count no of books loaned by each member
169  SELECT m.name AS member_name, COUNT(l.book_id) AS books_loaned
170  FROM members m
171  LEFT JOIN loans l ON m.member_id = l.member_id
172  GROUP BY m.name;

```

Output:

SQL ▾ < 1 / 1 > 1 - 5 of 5 		
title	author	member_name
The Great Gatsby	F. Scott Fitzgerald	Alice
To Kill a Mockingbird	Harper Lee	Bob
1984	George Orwell	Charlie
Pride and Prejudice	Jane Austen	David
Moby Dick	Herman Melville	Eve

SQL ▾ < 1 / 1 > 1 - 5 of 5 		
title	author	member_name
The Great Gatsby	F. Scott Fitzgerald	Alice
To Kill a Mockingbird	Harper Lee	Bob
1984	George Orwell	Charlie
Pride and Prejudice	Jane Austen	David
Moby Dick	Herman Melville	Eve

SQL ▼		< 1 / 1 > 1 - 5 of 5	
member_name	books_loaned		
Alice	1		
Bob	1		
Charlie	1		
David	1		
Eve	1		

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - o Retrieve all orders by a user.
 - o Find the most purchased product.
 - o Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

```

181 --create tables users,products,orders,orderdetails use normalization improvements and query optimization techniques
182 CREATE TABLE IF NOT EXISTS users (
183     user_id INT PRIMARY KEY,
184     username VARCHAR(50) NOT NULL UNIQUE,
185     email VARCHAR(100) NOT NULL UNIQUE,
186     password VARCHAR(255) NOT NULL
187 );
188 CREATE TABLE IF NOT EXISTS products (
189     product_id INT PRIMARY KEY,
190     product_name VARCHAR(100) NOT NULL,
191     price DECIMAL(10, 2) NOT NULL
192 );
193 CREATE TABLE IF NOT EXISTS orders (
194     order_id INT PRIMARY KEY,
195     user_id INT,
196     order_date DATE NOT NULL,
197     FOREIGN KEY (user_id) REFERENCES users(user_id)
198 );
199 CREATE TABLE IF NOT EXISTS order_details (
200     order_detail_id INT PRIMARY KEY,
201     order_id INT,
202     product_id INT,
203     quantity INT NOT NULL,
204     FOREIGN KEY (order_id) REFERENCES orders(order_id),
205     FOREIGN KEY (product_id) REFERENCES products(product_id)
206 );
207 --insert sample data into all tables

```

```

207 --insert sample data into all tables
208 INSERT INTO users (user_id, username, email, password) VALUES
209 (1, 'alice', 'alice@example.com', 'password1'),
210 (2, 'bob', 'bob@example.com', 'password2'),
211 (3, 'charlie', 'charlie@example.com', 'password3');
212 INSERT INTO products (product_id, product_name, price) VALUES
213 (1, 'Laptop', 999.99),
214 (2, 'Smartphone', 499.99),
215 (3, 'Headphones', 199.99);
216 INSERT INTO orders (order_id, user_id, order_date) VALUES
217 (1, 1, '2024-07-01'),
218 (2, 2, '2024-07-02'),
219 (3, 3, '2024-07-03');
220 INSERT INTO order_details (order_detail_id, order_id, product_id, quantity) VALUES
221 (1, 1, 1, 1),
222 (2, 1, 3, 2),
223 (3, 2, 2, 1),
224 (4, 3, 1, 1),
225 (5, 3, 2, 2);

```

```

226 --retrive all orders by a user
227 SELECT u.username, o.order_date, p.product_name, od.quantity
228 FROM orders o
229 JOIN users u ON o.user_id = u.user_id
230 JOIN order_details od ON o.order_id = od.order_id
231 JOIN products p ON od.product_id = p.product_id;
232 --find the most purchased product
233 SELECT p.product_name, SUM(od.quantity) AS total_quantity
234 FROM order_details od
235 JOIN products p ON od.product_id = p.product_id
236 GROUP BY p.product_name
237 ORDER BY total_quantity DESC
238 LIMIT 1;
239 --calculate total revenue in a given month
240 SELECT strftime('%Y-%m', o.order_date) AS month, SUM(p.price * od.quantity) AS total_revenue
241 FROM orders o
242 JOIN order_details od ON o.order_id = od.order_id
243 JOIN products p ON od.product_id = p.product_id
244 WHERE o.order_date >= '2024-07-01' AND o.order_date < '2024-08-01'
245 GROUP BY month;
246

```

Output:

SQL ▾				<	1	/ 1	>	1 - 5 of 5					
username	order_date	product_name	quantity										
alice	2024-07-01	Laptop	1										
alice	2024-07-01	Headphones	2										
bob	2024-07-02	Smartphone	1										
charlie	2024-07-03	Laptop	1										
charlie	2024-07-03	Smartphone	2										

SQL ▾		<	1	/ 1	>	1 - 1 of 1					
product_name	total_quantity										
Smartphone	3										

SQL ▾		<	1	/ 1	>	1 - 1 of 1					
month	total_revenue										
2024-07	3899.93										

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
 - o Insert examination results for a student.
 - o Retrieve subject-wise marks for a student.
 - o Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Screenshots:

```
252 --create tables named as students and subjects and exams and results and Define primary keys, foreign keys, and referen
253 CREATE TABLE IF NOT EXISTS students (
254     student_id INT PRIMARY KEY,
255     name VARCHAR(50) NOT NULL,
256     age INT NOT NULL
257 );
258 CREATE TABLE IF NOT EXISTS subjects (
259     subject_id INT PRIMARY KEY,
260     subject_name VARCHAR(50) NOT NULL
261 );
262 CREATE TABLE IF NOT EXISTS exams (
263     exam_id INT PRIMARY KEY,
264     subject_id INT,
265     exam_date DATE NOT NULL,
266     FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
267 );
268 CREATE TABLE IF NOT EXISTS results (
269     result_id INT PRIMARY KEY,
270     student_id INT,
271     exam_id INT,
272     score DECIMAL(5, 2) NOT NULL,
273     FOREIGN KEY (student_id) REFERENCES students(student_id),
274     FOREIGN KEY (exam_id) REFERENCES exams(exam_id)
275 );
```

```
276 --insert sample data into all tables
277 INSERT INTO students (student_id, name, age) VALUES
278 (1, 'Alice', 20),
279 (2, 'Bob', 22),
280 (3, 'Charlie', 19),
281 (4, 'David', 21),
282 (5, 'Eve', 20);
283 INSERT INTO subjects (subject_id, subject_name) VALUES
284 (1, 'Mathematics'),
285 (2, 'Physics'),
286 (3, 'Chemistry'),
287 (4, 'Biology'),
288 (5, 'History');
289 INSERT INTO exams (exam_id, subject_id, exam_date) VALUES
290 (1, 1, '2024-07-01'),
291 (2, 2, '2024-07-02'),
292 (3, 3, '2024-07-03'),
293 (4, 4, '2024-07-04'),
294 (5, 5, '2024-07-05');
```

```

296  --retrive  subject-wise marks for a student
297  SELECT s.name AS student_name, sub.subject_name, r.score
298  FROM results r
299  JOIN students s ON r.student_id = s.student_id
300  JOIN exams e ON r.exam_id = e.exam_id
301  JOIN subjects sub ON e.subject_id = sub.subject_id
302  WHERE s.student_id = 1;
303  --calculate avg marks for each subject
304  SELECT sub.subject_name, AVG(r.score) AS average_score
305  FROM results r
306  JOIN exams e ON r.exam_id = e.exam_id
307  JOIN subjects sub ON e.subject_id = sub.subject_id
308  GROUP BY sub.subject_name;
309

```

Output:

SQL ▾ < 1 / 1 > 1 - 2 of 2    

student_name	subject_name	score
Alice	Mathematics	88.5
Alice	Physics	91

SQL ▾ < 1 / 1 > 1 - 5 of 5    

subject_name	average_score
Biology	79.0
Chemistry	84.5
History	92.0
Mathematics	82.25
Physics	91.0